🛠️ 영남대학교 Agentic AI 강의 과정

서버에 Claude Code 설치 & AI 에이전트 실습 (2주차 - 4일차 실습)

대상: Claude를 처음 쓰는 입문자 학생

일정: 2026. 11. 08(일) 16:00 ~ 17:30 실습 · 17:30 ~ 18:00 정리(서버 이미지 루틴)

진행: 신인철 강사 (클라우드스퀘어)

> **"서버에 접속하고, Claude Code를 설치하고, AI에게 서버 일을 맡겨 본다!"**

🕐 4일차 — 서버 위의 AI 에이전트, Claude Code

2026. 11. 08(일) 13:00 ~ 18:00 시간표

이론 3시간 · 실습 1.5시간 · 정리 30분 · 50분 수업 + 10분 휴식 · 토 · 일 13:00 ~ 18:00 · 총 8일 / 40시간

시간	구분	내용
13:00 ~ 13:50	이론 1	3일차 회고 · 왜 서버에서 Claude Code인가 · 에이전트 동작 원리
13:50 ~ 14:00	휴식	☕ 10분 휴식
14:00 ~ 14:50	이론 2	접속 구조(Remote SSH · MobaXterm · Mac 터미널) · 설치 · 첫 실행 · 팀 계정 로그인
14:50 ~ 15:00	휴식	☕ 10분 휴식
15:00 ~ 15:50	이론 3	CLAUDE.md 이해 · 권한과 안전 규칙 · 서버 이미지 복구 · GitHub
15:50 ~ 16:00	휴식	☕ 10분 휴식
16:00 ~ 16:50	실습 ①	접속 → Claude Code 설치 → 팀 계정 로그인 (Lab 00~03)
16:50 ~ 17:00	휴식	☕ 10분 휴식
17:00 ~ 17:30	실습 ②	안전 규칙 (Lab 04) → Claude Code로 하루 할 일 메모 앱 v1~v4 (Lab 05)
17:30 ~ 18:00	정리	메모 앱 v3~v4 마무리 (~17:45) → 서버 이미지 생성 · 정리 (Lab 07) · Q&A — v5~v6 · GitHub(Lab 06)는 과제

🗺️ Day 4 실습 전체 로드맵 (Lab 00 ~ Lab 08)

준비 → 설치 → 바이브 코딩(메모 앱) → 백업 루틴

1단계 준비 · 접속

Lab 00
서버 상태 확인 · 기록 카드
필수

Lab 01
VS Code Remote SSH / MobaXterm / Mac 터미널 접속 · 파일 주고받기
필수

2단계 설치 · 로그인

Lab 02
Claude Code 설치
필수

Lab 03
팀 계정 로그인
필수

3단계 바이브 코딩

Lab 04
안전 규칙 CLAUDE.md · 권한 모드
필수

Lab 05
하루 할 일 메모 앱 v1~v6 달력 연동 · 서버 저장
필수 v1~v4

Lab 06
GitHub Push
과제

4단계 백업 · 복구

Lab 07
종료 30분 전 서버 이미지 생성
필수

Lab 08
서버 복구 절차
참고

⚠️ 실습 시작 전 필수 점검 (Checklist)

1. **Day 1 실습 기록 카드** 준비
   - 서버 공인 IP · root 비밀번호 · `yau-key-01.pem`
2. **팀(Team) 계정 초대 메일 수락**
   - 메일의 초대 링크로 가입 · 수락 완료
3. **VS Code 설치** + 확장 **Remote - SSH**
   - Windows는 **MobaXterm**도 가능
   - macOS · Linux는 기본 **터미널(`ssh`)** 도 가능
4. 브라우저: Chrome 또는 Edge
5. **스마트폰** (메모 앱 v1 한계 · v4 확인용)

🚫 **실습 서버에 두지 말 것**
개인 정보 · 비밀번호 파일 · 중요한 원본 자료. 오늘 서버는 **AI 에이전트가 root 권한으로 작업하는 연습장**입니다.

💡 **TIP**: 막히면 **슬라이드 번호 + 에러 화면**을 강사/조교에게 보여주세요.

📝 Day 4 실습 기록 카드 (메모장에 복사해서 채우기)

```
==================== Day 4 실습 기록 카드 ====================
1) 서버 이름            : yau-web-svr-01
2) 서버 공인 IP         : ___.___.___.___          ← Day 1 기록 카드
3) root 비밀번호        : (Day 1 기록 카드 참고 · 공유 금지)
4) 접속 도구            : VS Code Remote SSH / MobaXterm / 터미널
5) 작업 폴더            : /root/day4-lab
6) Claude Code 버전     : ____________             ← Lab 02
7) 로그인 계정 · 조직 : ____________           ← Lab 03 (/status)
8) 서버 이미지 이름     : yau-web-img-day4       ← Lab 07
9) (선택) GitHub 주소 : https://github.com/____/day4-lab  ← Lab 06
10) Day 3 달력 주소     : http://___.___.___.___/calendar.html ← Day 3 Lab 05
11) 메모 앱 주소        : http://___.___.___.___/memo.html     ← Lab 05
============================================================
```

서버를 복구해 **공인 IP가 바뀌면** 2)번을 새 IP로 고쳐 씁니다. (Lab 08)

💻 [Lab 00] 서버 상태 확인

Day 1~3에서 사용한 서버가 운영중인지 확인합니다

필수 · 5분

STEP 0-1. 콘솔에서 서버 · 공인 IP 확인

1. 콘솔 로그인 → Region `Korea` · Platform `VPC` 확인
2. 메뉴: `Services > Compute > Server > Server`
3. `yau-web-svr-01` 상태 확인

- **운영중** → 다음 단계로
- **정지** → 체크 후 **[서버 시작]** → 운영중까지 대기

4. 메뉴: `Server > Public IP` → 서버에 연결된 **공인 IP**가 기록 카드와 같은지 확인
5. 브라우저로 확인

- `http://<공인 IP>` → Day 2 웹페이지
- `http://<공인 IP>/calendar.html` → Day 3 달력

✅ **결과 확인**: 서버 **운영중** + 웹페이지 정상 표시

[스크린샷: 콘솔 Server 목록 (yau-web-svr-01 운영중) 화면]

🔌 [Lab 01] 서버 접속 & 파일 주고받기

VS Code Remote SSH (권장) · MobaXterm (Windows) · 터미널 ssh (Mac · Linux)

필수 · 15분

STEP 1-1. VS Code에 Remote - SSH 확장 설치

1. VS Code 실행 → 왼쪽 **확장(Extensions)** 아이콘 (`Ctrl+Shift+X` / Mac `⌘+Shift+X`)
2. 검색창에 **`Remote - SSH`** 입력
3. 게시자가 **Microsoft** 인 확장 → **[Install]**
4. 설치 후 왼쪽 아래 **초록/파란색 `><` 아이콘**이 생긴 것 확인

✅ **결과 확인**: 왼쪽 아래에 원격 연결 아이콘 표시

[스크린샷: VS Code 확장 검색 Remote - SSH 설치 화면]

STEP 1-2. 서버에 접속하고 작업 폴더 열기

1. `F1` (또는 `Ctrl/⌘+Shift+P`) → **`Remote-SSH: Connect to Host...`**
2. **`+ Add New SSH Host...`** → 아래 입력 후 Enter

```
ssh root@<공인 IP>
```

3. 설정 파일 위치 질문 → 첫 번째 항목 선택 → **[Connect]**
4. 플랫폼 질문 → **`Linux`** / 지문(fingerprint) 질문 → **`Continue`**
5. 비밀번호 입력창 → **root 비밀번호 붙여넣기** → Enter
6. 왼쪽 아래에 **`SSH: <공인 IP>`** 표시 확인
7. `Ctrl + ` ` 로 터미널 열기 → `root@yau-web-svr-01:~#` 확인

[스크린샷: VS Code Remote-SSH 접속 완료 (왼쪽 아래 SSH 표시) 화면]

STEP 1-3. (Windows 대안) MobaXterm으로 접속

1. MobaXterm 실행 → 왼쪽 위 **[Session]** → **[SSH]**
2. **Remote host**: `<공인 IP>`
3. **Specify username** 체크 → `root`
4. **[OK]** → 비밀번호 입력 (화면에 안 보이는 것이 정상)
5. 비밀번호 저장 질문은 **No** 권장 (공용 PC일 때)

✅ **결과 확인**: 오른쪽 터미널에 `root@yau-web-svr-01:~#`, 왼쪽에 **SFTP 파일 창** 표시

[스크린샷: MobaXterm SSH 세션 설정 화면]

STEP 1-3B. (macOS · Linux 대안) 터미널에서 `ssh` 로 접속

① 터미널 열기

- macOS: `⌘ + Space` → **"터미널"** → Enter
- Linux: `Ctrl + Alt + T`

② **접속** (별도 설치 없이 바로 사용)

```
ssh root@<공인 IP>
```

1. 처음 접속 시 `Are you sure ... (yes/no)?` → **`yes`**
2. `password:` → root 비밀번호 **붙여넣기**(`⌘+V`) → Enter (화면에 안 보이는 것이 정상)
3. 끝낼 때: `exit`

✅ **결과 확인**: `root@yau-web-svr-01:~#` 표시

③ **파일 주고받기** (Drag & Drop 대신 `scp`) — **내 Mac 터미널**에서 실행

```
# 내 Mac → 서버 (업로드)
scp ~/Desktop/test.png root@<공인 IP>:/root/day4-lab/

# 서버 → 내 Mac (다운로드)
scp root@<공인 IP>:/root/day4-lab/test.png ~/Downloads/
```

💡 **TIP**: Mac에서도 **VS Code + Remote - SSH**가 가장 편합니다. (`F1` 대신 `⌘ + Shift + P`) 터미널 `ssh`는 VS Code 연결이 안 될 때의 **대안**입니다.

STEP 1-4. 작업 폴더 만들기 & 파일 Drag & Drop

1. 서버 터미널에서 실습 폴더 생성

```
mkdir -p /root/day4-lab && cd /root/day4-lab && pwd
```

2. **VS Code**: `File > Open Folder...` → `/root/day4-lab` 입력 → **[OK]** (비밀번호 다시 입력할 수 있음)
3. 내 PC의 아무 파일(예: 이미지 1개)을 VS Code **탐색기**로 끌어다 놓기 → **업로드**
4. 탐색기의 파일 **우클릭 → Download...** → 내 PC로 **다운로드**

- MobaXterm: 왼쪽 SFTP 창 주소를 `/root/day4-lab` 으로 이동 → 끌어다 놓기 / 끌어오기
- macOS · Linux 터미널: `scp` 명령 (STEP 1-3B)

```
ls -l /root/day4-lab          # 업로드한 파일 확인
```

✅ **결과 확인**: 업로드한 파일이 `ls -l` 결과에 보이고, 다운로드도 성공

🚧 Lab 01 자주 막히는 지점

증상	원인	해결 방법
`Could not establish connection` / 시간 초과	서버 정지 / 공인 IP 변경 / ACG 22	Lab 00 다시 확인 · ACG `yau-web-acg` Inbound **22** 확인
비밀번호를 계속 다시 물어봄	비밀번호 오입력	기록 카드에서 **다시 복사** 후 한 번만 붙여넣기
`REMOTE HOST IDENTIFICATION HAS CHANGED`	같은 IP에 다른 서버(복구 등)	PC 터미널에서 `ssh-keygen -R <공인 IP>` 후 재접속
VS Code가 서버 설치 중 멈춤	서버에 VS Code Server 다운로드 중	1~2분 대기 · 실패 시 창 닫고 재접속
플랫폼을 잘못 골랐음	Windows/macOS 선택	`F1` → `Remote-SSH: Settings` 에서 플랫폼을 `linux` 로 수정
학교 네트워크에서 접속 불가	22번 포트 차단	휴대폰 핫스팟으로 재시도
(Mac) `Permission denied`	비밀번호 오입력 · 계정 이름	`ssh root@<공인 IP>` 의 **root@** 확인 · 비밀번호 다시 붙여넣기 (`⌘+V`)

📦 [Lab 02] Claude Code 설치

서버(Ubuntu)에만 설치합니다 — 학생 PC에는 설치하지 않아요

필수 · 10분

STEP 2-1. 설치 명령 실행

서버 터미널(`root@yau-web-svr-01`)에서 실행합니다.

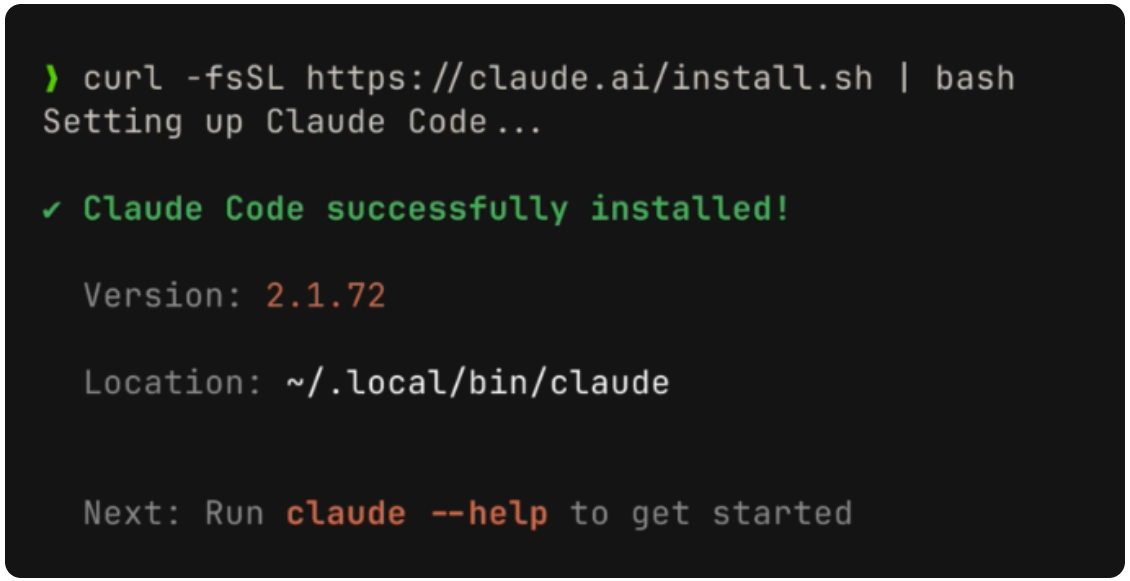
```
# 1. 사전 점검: 필요한 도구 설치 (이미 있으면 건너뜀)
apt-get update -y && apt-get install -y curl git

# 2. Claude Code 설치 (네이티브 설치 스크립트)
curl -fsSL https://claude.ai/install.sh | bash
```

```
# 3. 새로 설치된 명령을 바로 쓰도록 쉘 설정 다시 읽기
source ~/.bashrc
```

⚠️ **강의 전 확인**: 설치 명령은 버전에 따라 바뀔 수 있습니다. 수업 전에 **Claude Code 공식 문서의 최신 설치 명령**으로 이 슬라이드를 맞춰 주세요.

STEP 2-2. 설치 성공 화면 확인하기



📋 설치 명령 (복사해서 붙여넣기)

```
curl -fsSL https://claude.ai/install.sh | bash
```

> 직접 타이핑하지 말고 **복사 → 붙여넣기** 하세요. 오타가 나면 설치가 실패합니다.

1. `✔ Claude Code successfully installed!` 문구 확인
2. `Version:` 숫자 → **기록 카드 6)** 에 기록
   - 화면의 `2.1.72` 는 예시이며 수업 당일 버전은 다를 수 있습니다.
3. `Location: ~/.local/bin/claude` 확인
   - root 계정이면 실제 위치는 `/root/.local/bin/claude`
4. 다음 단계 안내(`claude --help`)가 나오면 설치 완료
   - 바로 실행이 안 되면 `source ~/.bashrc`

✅ **결과 확인**: 설치 성공 문구 + 버전 · 위치 표시

STEP 2-3. 설치 확인 & 진단

1. 버전 확인

```
claude --version
```

2. 설치 상태 진단

```
claude doctor
```

✅ **결과 확인**: 버전 번호가 출력되면 설치 성공 → **기록 카드 6)** 에 버전 기록

(대안) npm으로 설치하기 — 네이티브 설치가 안 될 때만

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -   # Node.js 20 저장소 추가
apt-get install -y nodejs && node -v                        # v20 이상인지 확인
npm install -g @anthropic-ai/claude-code
```

> 💡 Ubuntu 22.04 기본 저장소의 `nodejs` 는 **v12**라서 Claude Code(Node.js 18 이상 필요)가 설치되지 않습니다. 그래서 NodeSource 저장소에서 새 버전을 설치합니다.

STEP 2-3. 설치 확인 & 진단 — npm 대안 설치 (확대 보기)

(대안) npm으로 설치하기 — 네이티브 설치가 안 될 때만

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs && node -v
npm install -g @anthropic-ai/claude-code
```

💡 Ubuntu 22.04 기본 저장소의 `nodejs` 는 **v12** → Claude Code(Node.js 18 이상 필요) 설치 실패. 그래서 NodeSource 저장소에서 **v20**을 설치합니다.

🚧 Lab 02 자주 막히는 지점

증상	원인	해결 방법
`claude: command not found`	설치 경로(`~/.local/bin`)가 PATH에 없음	`source ~/.bashrc` → 안 되면 `export PATH="$HOME/.local/bin:$PATH"` 또는 터미널을 닫고 다시 열기
`curl: command not found`	curl 미설치	`apt-get install -y curl`
`E: Could not get lock ...`	서버 부팅 직후 자동 업데이트 중	1~2분 기다린 뒤 다시 실행
설치 스크립트 다운로드 실패	네트워크 일시 오류	명령 다시 실행 · 계속 실패 시 npm 대안
`claude doctor` 경고	설정 · 버전 안내 메시지	메시지 내용을 강사에게 보여주기 (대부분 진행 가능)

🔑 [Lab 03] 팀 계정으로 로그인

서버엔 브라우저가 없으니, 인증은 내 PC 브라우저에서!

필수 · 10분

STEP 3-1. Claude Code 실행 & 로그인 방식 선택

1. 작업 폴더에서 Claude Code 실행

```
cd /root/day4-lab
claude
```

2. 화면 테마 선택 → 원하는 색 선택 후 Enter
3. 로그인 방식 선택 화면

- **Claude 계정(구독)으로 로그인** 선택 ✅
- API 키(Console) 방식은 선택하지 않음

4. 터미널에 **인증 URL**이 표시됩니다.

[스크린샷: Claude Code 로그인 방식 선택 화면]

STEP 3-2. 내 PC 브라우저에서 승인 → 코드 붙여넣기

① 서버 터미널
URL 복사
표시된 긴 URL 전체 선택 · 복사

→

② 내 PC 브라우저
URL 열기
주소창에 붙여넣기

→

③ 브라우저
팀 계정 로그인
초대받은 이메일로 로그인 · 조직 선택 · 승인

→

④ 브라우저
인증 코드 복사
화면에 표시된 코드 복사

→

⑤ 서버 터미널
코드 붙여넣기
Enter → 로그인 완료

- VS Code 터미널: URL을 `Ctrl/⌘ + 클릭` 하면 내 PC 브라우저가 바로 열립니다.
- MobaXterm: 마우스로 URL 드래그 → 자동 복사 → 브라우저에 붙여넣기
- Mac 터미널: URL 드래그 → `⌘ + C` → 브라우저에 붙여넣기 (또는 `⌘ + 클릭`)

[스크린샷: 브라우저 Claude 인증 승인 · 코드 표시 화면]

STEP 3-3. 로그인 상태 확인

1. Claude Code 입력창에서 실행

```
/status
```

2. 표시 내용 확인

- 로그인 방식: **Claude 계정(구독)**
- 계정 이메일 · **조직(팀 이름)**
- 모델 · 버전

3. 첫 인사 해보기

```
안녕! 지금 이 서버의 운영체제와 현재 폴더 위치를 알려줘.
```

4. Claude가 명령 실행 승인을 요청하면 → 명령을 읽고 **Yes**

✅ **결과 확인**: `/status` 에 팀 조직이 표시되고, 첫 질문에 답이 오면 성공 → **기록 카드 7)** 기록

🚧 Lab 03 자주 막히는 지점

증상	원인	해결 방법
조직(팀)이 보이지 않음	팀 초대 미수락 / 다른 이메일로 로그인	초대 메일 수락 → `/logout` 후 `/login` 으로 다시 로그인
코드 붙여넣기 후 오류	코드 일부만 복사 / 시간 초과	`/login` 으로 처음부터 다시
URL이 두 줄로 잘려 복사됨	터미널 줄바꿈	터미널 창을 넓히거나 URL 전체를 정확히 선택
API 키 방식으로 로그인됨	로그인 방식 선택 실수	`/logout` → `/login` → **Claude 계정(구독)** 선택
사용량 제한 안내	팀 좌석 사용 한도	강사에게 알리기 · 잠시 후 재시도

🛡️ [Lab 04] 안전 규칙 & 권한 모드

AI 에이전트에게 "업무 지침서"를 먼저 줍니다

필수 · 10분

STEP 4-1. CLAUDE.md 안전 규칙 만들기

> Claude Code를 잠시 종료(`/exit`)하고 서버 터미널에서 아래 블록 **전체를 복사 → 붙여넣기 → Enter**

```
cat << 'EOF' > /root/day4-lab/CLAUDE.md
# Day 4 실습 서버 규칙 (yau-web-svr-01)

## 반드시 지킬 것
- 파일을 수정하기 전에 같은 폴더에 백업본을 만든다. (예: index_before_day4.html)
- 명령을 실행하기 전에 무엇을 하는 명령인지 한국어로 한 줄 설명한다.
- 작업이 끝나면 변경한 파일 목록과 확인 방법을 알려준다.

## 절대 하지 말 것
- /var/www/html/index_day1.html 수정 · 삭제 금지 (Day 1 백업본)
- rm -rf, 대량 삭제, 디스크 포맷 금지
- SSH 설정(/etc/ssh), 방화벽, 사용자 계정 · 비밀번호 변경 금지
EOF
cat /root/day4-lab/CLAUDE.md
```

✅ **결과 확인**: 규칙 내용이 그대로 출력되면 성공

STEP 4-1. CLAUDE.md 안전 규칙 만들기 — 규칙 파일 명령 (확대 보기)

```
cat << 'EOF' > /root/day4-lab/CLAUDE.md
# Day 4 실습 서버 규칙 (yau-web-svr-01)

## 반드시 지킬 것
- 파일을 수정하기 전에 같은 폴더에 백업본을 만든다. (예: index_before_day4.html)
- 명령을 실행하기 전에 무엇을 하는 명령인지 한국어로 한 줄 설명한다.
- 작업이 끝나면 변경한 파일 목록과 확인 방법을 알려준다.

## 절대 하지 말 것
- /var/www/html/index_day1.html 수정 · 삭제 금지 (Day 1 백업본)
- rm -rf, 대량 삭제, 디스크 포맷 금지
- SSH 설정(/etc/ssh), 방화벽, 사용자 계정 · 비밀번호 변경 금지
EOF
cat /root/day4-lab/CLAUDE.md
```

STEP 4-2. 규칙이 적용되는지 확인

1. 다시 Claude Code 실행

```
cd /root/day4-lab && claude
```

2. 아래처럼 물어보기

```
이 폴더의 CLAUDE.md 규칙을 요약해줘. 그리고 /var/www/html/index_day1.html 을 지워달라고 하면 어떻게 할 거야?
```

3. Claude가 **규칙을 근거로 거절하거나 확인**하는지 관찰합니다.

4. `/memory` 입력 → 지금 적용 중인 `CLAUDE.md` 파일을 확인 (`Esc` 로 닫기)

> ❗ **IMPORTANT**: `CLAUDE.md`는 강력한 **지침**이지만 완벽한 차단벽은 아닙니다. 최종 안전장치는 **사람의 승인**과 **서버 이미지**입니다.

STEP 4-3. 권한 모드 확인하기

1. 입력창에서 `Shift + Tab` 을 눌러가며 입력창 아래 표시 변화 관찰

- 기본 (승인 요청) → 수정 자동 허용 → 계획 모드(Plan)

2. 오늘은 **기본 모드(승인 요청)** 로 돌아와서 진행합니다.

3. `/permissions` 입력 → 허용 · 차단 규칙 화면 구경 (변경하지 않고 `Esc`)

승인 화면 선택지	의미
Yes	이번 한 번 실행
Yes, 이번 세션 동안 허용	같은 종류는 묻지 않음
No (다른 지시)	거절하고 다른 방법 지시

> 💡 모르는 명령은 **No** 후 "이 명령이 뭘 하는지 설명해줘"

📝 [Lab 05] Claude Code로 바이브 코딩 — 하루 할 일 메모 앱

Day 3 달력과 연동하고, 서버에 데이터를 남겨 새로고침해도 사라지지 않게!

필수 · v1~v4 (약 35분 · 정리 시간 앞 15분까지 사용)

과제 · v5~v6

STEP 5-0. 오늘 만들 앱 — 6단계로 점점 완성도 있게

v1 · 필수
메모 화면
`memo.html`
추가 · 체크 · 삭제 (브라우저 저장)

→

v2 · 필수
달력 연동
Day 3
`calendar.html`
날짜 클릭 → 그날 메모

→

v3 · 필수
서버 API
Python Flask
`memos.json`
에 저장

→

v4 · 필수
Nginx 연결
화면 ↔ 서버 저장
PC · 폰 어디서나 같은 메모

→

v5 · 과제
서비스 등록
systemd
재부팅해도 동작

→

v6 · 과제
완성도
달력에 개수 표시
완료율 · 백업 · 비밀번호

- **방법**: 각 단계의 프롬프트를 Claude Code에 **복사 → 붙여넣기** → 승인 요청을 **읽고** 승인 → **내가 직접** 브라우저 · 터미널로 확인
- **원칙**: 한 번에 다 만들지 않고 **작게 만들고 → 확인하고 → 다음 단계** (Day 2의 바이브 코딩과 같은 리듬)

STEP 5-0. 완성 구조 미리 보기 & 메모 앱 규칙 추가



① 시작 전 규칙 추가 (서버 터미널, Claude Code 밖에서)

```
cat << 'EOF' >> /root/day4-lab/CLAUDE.md

## 메모 앱 규칙 (Lab 05)
- 화면: /var/www/html/memo.html
- API: /root/day4-lab/memo-api/app.py
  (127.0.0.1:5000 에서만 실행)
- 데이터: /var/lib/memo-app/memos.json
- 기존 파일 수정 전 *_before_day4 백업
- Nginx 설정 수정 후 nginx -t 로 검사
- ACG · 방화벽 · 포트 변경 금지
EOF
```

② `cd /root/day4-lab && claude` 로 다시 시작

⚠️ **로그인 없는 메모장**입니다. 공인 IP를 아는 사람은 누구나 볼 수 있으니 **개인 정보 · 비밀번호는 절대 쓰지 마세요.** (비밀번호 보호는 v6 선택)

STEP 5-0. 완성 구조 미리 보기 (확대 보기)

🌐 브라우저 (PC · 스마트폰)

`/calendar.html` · `/memo.html`

▼ HTTP 80 (ACG 그대로)

🟩 Nginx

화면 파일 제공 · `/api/` 요청은 안쪽으로 전달

▼ 127.0.0.1:5000 (서버 안에서만)

🐍 메모 API (Flask)

`/root/day4-lab/memo-api/app.py`

▼ 읽기 · 쓰기

💾 데이터 파일

`/var/lib/memo-app/memos.json`

STEP 5-0. 메모 앱 규칙 추가 (확대 보기)

① 서버 터미널에서 (Claude Code 밖에서) 복사 → 붙여넣기 → Enter

```
cat << 'EOF' >> /root/day4-lab/CLAUDE.md

## 메모 앱 규칙 (Lab 05)
- 화면: /var/www/html/memo.html
- API: /root/day4-lab/memo-api/app.py
  (127.0.0.1:5000 에서만 실행)
- 데이터: /var/lib/memo-app/memos.json
- 기존 파일 수정 전 *_before_day4 백업
- Nginx 설정 수정 후 nginx -t 로 검사
- ACG · 방화벽 · 포트 변경 금지
EOF
```

② `cd /root/day4-lab && claude` 로 다시 시작

STEP 5-1. v1 — 하루 할 일 메모 화면 만들기

프롬프트 (복사 → 붙여넣기)

```
/var/www/html/memo.html 에 "하루 할 일 메모" 페이지를 만들어줘.
- 상단에 오늘 날짜 표시
- 할 일 입력칸 + [추가] 버튼 (Enter 키도 동작)
- 목록: 완료 체크박스(체크하면 줄 긋기) · 삭제 버튼
- 데이터는 브라우저 localStorage 에 날짜별로 저장
- HTML · CSS · JS 를 한 파일에, 외부 라이브러리 없이
- 스마트폰에서도 보기 좋게, Day 3 달력과 어울리는 디자인
만들기 전에 계획을 먼저 보여줘.
```

진행

1. Claude가 보여주는 **계획**을 읽고 진행 승인
2. 파일 생성 승인 (`/var/www/html/memo.html`)
3. 브라우저에서 확인 → **기록 카드 11)**

```
http://<공인 IP>/memo.html
```

✅ 결과 확인

- 할 일 3개 추가 → 1개 체크 → 1개 삭제
- **F5 새로고침해도 남아 있음**

> 💡 마음에 안 들면 **같은 대화에서** "버튼을 더 크게", "완료한 일은 아래로" 처럼 이어서 요청

STEP 5-2. v1의 한계 체험 — "폰으로 열면 메모가 없다?!"

1. **스마트폰**으로 `http://<공인 IP>/memo.html` 접속
2. PC에서 쓴 할 일이 **보이나요? → 😮 비어 있음**
3. 다른 브라우저(Edge ↔ Chrome)로 열어도 마찬가지
4. Claude Code에 물어보기

```
memo.html 의 데이터는 정확히 어디에 저장되고 있어?
왜 스마트폰에서는 안 보이는지 초보자에게 설명해줘.
```

구분	v1 · localStorage	v3~v4 · 서버 저장
저장 위치	**내 브라우저 안**	**서버의 파일**
새로고침	남아 있음	남아 있음
다른 기기 · 브라우저	❌ 안 보임	✅ 같은 메모
브라우저 기록 삭제	❌ 사라짐	✅ 그대로
서버 이미지 백업	❌ 포함 안 됨	✅ 함께 백업

🎯 그래서 **진짜로 "서버에 데이터를 남기는"** v3~v4가 필요합니다.

STEP 5-3. v2 — Day 3 달력과 연동하기

프롬프트

```
/var/www/html/calendar.html 을 수정하기 전에
calendar_before_day4.html 로 백업해줘.
그 다음
1) 달력에서 날짜를 클릭하면
   memo.html?date=2026-11-08 형식으로 이동
2) memo.html 은 주소의 date 값을 읽어 그 날짜의
   메모를 보여주고, date 가 없으면 오늘 날짜
3) memo.html 상단에 [◀ 전날] [다음 날 ▶] [📅 달력] 버튼
4) index.html 메뉴에 "달력" "메모" 버튼 추가
수정한 파일 목록과 확인할 주소를 알려줘.
```

진행 · 관찰 포인트

- **백업 파일을 먼저** 만드는지 (CLAUDE.md 규칙)
- 변경 내용(diff)을 읽고 승인
- Day 3 달력 디자인이 **유지**되는지

✅ 결과 확인

1. `http://<공인 IP>` → **달력** 버튼
2. 달력에서 **11월 8일** 클릭 → 11/8 메모 화면
3. 할 일 추가 → **◀ 전날** → 11/7은 비어 있음
4. **다음 날 ▶** → 11/8 할 일이 그대로

> 💡 Day 3에 **Cowork(내 PC)** 가 만든 파일을, 오늘 **Claude Code(서버)** 가 이어서 고칩니다.

STEP 5-4. v3 — 서버에 메모를 저장하는 API 만들기

프롬프트

```
메모를 서버에 저장하는 API 를 만들어줘.
- 패키지: apt 로 python3-flask 설치
- 코드: /root/day4-lab/memo-api/app.py
- 데이터: /var/lib/memo-app/memos.json
  (폴더 · 파일이 없으면 자동 생성)
- 127.0.0.1:5000 에서만 실행
- API
  GET    /api/memos?date=YYYY-MM-DD  그날 목록
  POST   /api/memos                  추가 {date, text}
  PATCH  /api/memos/<id>             완료 {done}
  DELETE /api/memos/<id>             삭제
- 저장할 때 파일이 깨지지 않게 임시 파일에 쓴 뒤 교체
지금은 터미널에서 직접 실행해 테스트만 하고,
화면(memo.html)은 아직 고치지 마.
```

새로 나온 용어

용어	뜻
API	프로그램끼리 데이터를 주고받는 **창구**
Flask	Python으로 API를 쉽게 만드는 도구
JSON	`{"text": "과제 제출"}` 처럼 쓰는 **데이터 형식**
127.0.0.1	"서버 자기 자신" — 바깥에서는 접속 불가

관찰 포인트

- `apt install` 승인 요청 → 무엇을 설치하는지 읽기
- Claude가 **스스로 실행하고 테스트**하는지

STEP 5-5. v3 확인 — `curl` 로 API에 직접 물어보기

① API 실행 (새 터미널 창에서, 이미 실행 중이면 생략)

```
cd /root/day4-lab/memo-api && python3 app.py
```

② 다른 터미널에서 테스트 (복사 → 붙여넣기)

```
curl -s -X POST http://127.0.0.1:5000/api/memos \
  -H "Content-Type: application/json" \
  -d '{"date":"2026-11-08","text":"서버 저장 테스트"}'

curl -s "http://127.0.0.1:5000/api/memos?date=2026-11-08"

cat /var/lib/memo-app/memos.json
```

✅ 결과 확인

- POST 결과로 `id` 가 붙은 메모가 돌아옴
- GET 결과에 `"서버 저장 테스트"` 가 보임
- `memos.json` **파일 안에 데이터가 실제로 저장**됨

이해 체크

- 이 데이터는 **브라우저가 아니라 서버의 파일**에 있습니다.
- 아직 브라우저(`/api/...`)로는 접속되지 않습니다.
  → v4에서 Nginx가 연결

> 💡 실행이 안 되면 오류 메시지를 그대로 Claude Code에 붙여넣고 **"원인과 해결 방법을 알려줘"**

STEP 5-6. v4 — Nginx로 연결하고 화면을 서버 저장으로 바꾸기

프롬프트

```
1) Nginx 설정에서 /api/ 로 오는 요청을
   127.0.0.1:5000 으로 전달(proxy_pass)하게 해줘.
   설정 파일은 수정 전에 백업하고,
   nginx -t 로 검사한 뒤 reload 해줘.
2) memo.html 을 localStorage 대신 /api/memos 를
   쓰도록 바꿔줘. 저장 중 · 오류 상태를 화면에
   짧게 표시해줘. calendar.html 연동은 그대로 유지.
3) 브라우저에서 확인할 순서를 알려줘.
```

📌 v3의 API가 **실행 중**이어야 합니다. (STEP 5-5 ①)

관찰 포인트

- `/etc/nginx/...` 수정 전에 **백업**하는지
- `nginx -t` 결과가 **`test is successful`** 인지
- ACG · 포트를 건드리지 않는지 (80번 그대로)

✅ **결과 확인** — 오늘의 핵심!

1. PC에서 11/8 할 일 추가 → **F5** → 그대로
2. **스마트폰**에서 같은 주소 → **같은 할 일이 보임** 🎉
3. 폰에서 체크 → PC에서 F5 → 체크 반영
4. 서버에서 `cat /var/lib/memo-app/memos.json`

❗ **v1과 무엇이 달라졌나요?** 데이터가 **내 브라우저 → 네이버클라우드 서버**로 옮겨 갔습니다. 이것이 "서버에 데이터를 남긴다"의 의미입니다.

STEP 5-7. v5 (과제) — 터미널을 닫아도 · 재부팅해도 동작하게

문제: v3에서 `python3 app.py` 를 실행한 터미널을 닫으면 API도 **꺼집니다.** → 메모 저장 실패

프롬프트

```
memo-api 를 systemd 서비스로 등록해줘.
- 서비스 이름: memo-api
- 서버가 부팅되면 자동 시작, 죽으면 자동 재시작
- 직접 실행 중인 python3 app.py 가 있으면 먼저 종료
- 등록 후 상태 확인 명령과 로그 보는 법을 알려줘
```

확인 명령 (복사 → 붙여넣기)

```
systemctl status memo-api --no-pager
systemctl restart memo-api
journalctl -u memo-api -n 20 --no-pager
```

✅ 결과 확인

- `active (running)` 표시
- 터미널을 모두 닫고 다시 접속해도 메모 저장 OK
- (선택) `reboot` → 1~2분 뒤 재접속 → 메모 그대로

> 💡 Nginx도 이렇게 **systemd 서비스**로 계속 실행되고 있었습니다.

STEP 5-8. v6 (과제 · 선택) — 완성도 높이기

A. 달력에 할 일 개수 표시

```
API 에 GET /api/summary?month=YYYY-MM 를 추가해
날짜별 (전체 개수, 완료 개수)를 돌려주고,
calendar.html 의 각 날짜에 작은 배지로 표시해줘.
```

B. 완료율 · 정렬

```
memo.html 상단에 오늘 완료율(%) 막대를 보여주고,
완료한 일은 목록 아래로 내려줘.
```

C. 데이터 백업

```
memos.json 을 /var/lib/memo-app/backup/ 에
날짜를 붙여 복사하는 명령을 알려주고 한 번 실행해줘.
```

D. 비밀번호 보호 (Nginx Basic Auth)

```
memo.html 과 /api/ 에 Nginx Basic Auth 를 걸어줘.
아이디는 student, 비밀번호는 내가 직접 입력할게.
설정 백업 · nginx -t 검사를 꼭 해줘.
```

💡 기능 하나 추가 → 확인 → 다음 기능. **한 번에 여러 개 요청하지 않기!**

STEP 5-9. Lab 05 정리 — 무엇을 만들었고, 무엇을 배웠나

단계	만든 것	배운 개념
v1	`memo.html` (localStorage)	프론트엔드 · 브라우저 저장의 한계
v2	달력 ↔ 메모 연동	주소의 `?date=` 값으로 화면 연결 · 기존 파일 백업 후 수정
v3	Flask API + `memos.json`	API · JSON · 서버에 데이터 저장
v4	Nginx `/api/` 연결	웹서버 ↔ 앱 연결 (리버스 프록시) · 어디서나 같은 데이터
v5	systemd `memo-api`	서비스로 계속 실행 · 로그 확인
v6	개수 배지 · 완료율 · 백업 · 비밀번호	기능을 작게 하나씩 늘리는 바이브 코딩

💡 **Lab 07과 연결**: `memos.json` 은 **서버 안의 파일**이므로, 오늘 만드는 **서버 이미지에 메모 데이터도 함께 백업**됩니다.

🚧 Lab 05 자주 막히는 지점

증상	원인	해결 방법
`memo.html` 이 404	다른 폴더에 생성	`ls /var/www/html/` · 경로를 정확히 다시 요청
폰에서 메모가 안 보임 (v1~v2)	localStorage는 기기별 저장 (정상)	v3~v4 진행
`curl` 결과 한글이 `\uc11c…` 로 보임	JSON의 한글 표기 방식 (정상)	그대로 진행 · 브라우저 화면에서는 한글로 보임
`curl` 연결 거부 (v3)	API 미실행	`python3 app.py` 실행 중인지 · v5 서비스 상태 확인
화면에 "저장 실패" (v4)	API 꺼짐 · Nginx 설정	`curl 127.0.0.1:5000/api/memos?date=2026-11-08` · `nginx -t`
`502 Bad Gateway`	Nginx는 OK, API가 꺼짐	API 다시 실행 · `systemctl status memo-api`
달력이 깨짐 (v2)	수정 실수	`calendar_before_day4.html` 로 복원 요청
승인 요청이 너무 많음	기본 모드 (정상)	반복 작업이면 "이번 세션 동안 허용"
대화가 길어져 느려짐	대화 기록 누적	`/compact` · 단계마다 `/clear` 후 새로 시작

🐙 [Lab 06] (과제) 작업물을 GitHub에 Push

서버 이미지는 서버를, GitHub는 내 작업물을 남깁니다

과제 · 메모 앱 코드도 함께 Push

STEP 6-1. GitHub 저장소 & 토큰 준비

1. `github.com` 로그인 → **[New repository]**
   - 이름: `day4-lab` / **Private** 권장 / 나머지 기본값 → **[Create]**
2. 토큰 만들기: `Settings > Developer settings > Personal access tokens`
   - 실습 저장소만 접근 · 짧은 만료 기간 권장
   - 생성된 토큰을 **한 번만** 복사 (다시 볼 수 없음)

> 🚫 토큰은 비밀번호와 같습니다. 채팅 · 프롬프트 · 파일에 적지 마세요.



STEP 6-2. Claude Code와 함께 커밋 & Push

1. Claude Code에 요청하기

```
/root/day4-lab 을 git 저장소로 만들고 .gitignore 를 만들어줘.
.pem, .env, 비밀번호가 들어간 파일은 제외해줘.
/var/www/html/index.html 과 about.html 을 web/ 폴더로 복사한 뒤 첫 커밋을 만들어줘.
push 는 내가 직접 할게.
```

2. 원격 저장소 연결 & Push (사람이 직접 실행)

```
cd /root/day4-lab
git remote add origin https://github.com/<내 아이디>/day4-lab.git
git branch -M main && git push -u origin main
# Username: GitHub 아이디 / Password: 토큰 붙여넣기
```

✅ **결과 확인**: GitHub 저장소 페이지에 `CLAUDE.md`, `web/` 폴더 표시 → **기록 카드 9)**

💾 [Lab 07] 종료 30분 전 루틴: 서버 이미지 생성

매일 같은 시간, 같은 절차 — 클라우드 운영의 백업 습관

필수 · 17:30 시작

STEP 7-1. 작업 정리 & 서버 이미지 생성

1. Claude Code 종료: `/exit`
2. 콘솔 메뉴: `Services > Compute > Server > Server`
3. `yau-web-svr-01` 체크 → **[서버 관리 및 설정 변경] → [내 서버 이미지 생성]**
4. 이미지 이름: **`yau-web-img-day4`**
5. **[생성]** 클릭

> 💡 VPC 환경은 서버가 **운영중**이어도 이미지를 만들 수 있습니다. (정지 여부는 강사 안내)

[스크린샷: 내 서버 이미지 생성 팝업 (yau-web-img-day4) 화면]

STEP 7-2. 이미지 상태 확인 & 오래된 이미지 정리

1. 메뉴: `Server > Server Image`
2. `yau-web-img-day4` 상태: `생성중` → **`생성됨`** 확인 (수 분 소요)
3. **오래된 이미지 정리**: 최근 **1~2개만** 남기고 이전 이미지 선택 → **[삭제]**
4. **기록 카드 8)** 에 이미지 이름 기록

✅ **결과 확인**: 목록에 `yau-web-img-day4` **생성됨**

[스크린샷: Server Image 목록 (생성됨 상태) 화면]

🚫 서버 · 공인 IP는 **반납하지 않습니다.** 다음 수업에서 계속 사용합니다.

🚑 [Lab 08] (참고) 서버가 망가졌을 때 복구하기

이미지로 새 서버를 만들고, 새 공인 IP로 접속합니다

참고 · 필요할 때 강사와 함께

STEP 8-1. 망가진 서버 정지 & 이미지로 새 서버 생성

① 정지

yau-web-svr-01

[서버 정지]

→

② 이미지 선택

Server Image

`yau-web-img-day4`

→ [서버 생성]

→

③ 서버 설정

Day 1과 동일

VPC

`yau-vpc-01`

Subnet

`yau-pub-sub-01`

→

④ 이름 · 키 · ACG

yau-web-svr-02

인증키

`yau-key-01`

ACG

`yau-web-acg`

- Subnet은 반드시 **Public** (`yau-pub-sub-01`)
- 서버 이름은 구분을 위해 **`yau-web-svr-02`**
- 상태가 **운영중**이 될 때까지 대기 (5~10분)

[스크린샷: 서버 이미지로 서버 생성 화면]

STEP 8-2. 새 공인 IP 연결 → 접속 → 옛 자원 반납

1. `Server > Public IP` → **[공인 IP 신청]** → 적용 서버 **`yau-web-svr-02`**
2. 새 서버 체크 → **[관리자 비밀번호 확인]** → `yau-key-01.pem` 업로드 → 비밀번호 확인
3. **기록 카드 2)** 를 **새 공인 IP**로 수정 → 접속 확인

```
ssh root@<새 공인 IP>
ls -l /var/www/html/ /root/day4-lab/ && claude --version
```

4. 브라우저 `http://<새 공인 IP>` 확인 → `claude` 실행 후 `/status` (로그인 유지 여부 확인, 필요 시 `/login`)
5. **옛 자원 반납** (요금 두 배 방지)

- 옛 공인 IP: [서버에서 할당 해제] → **[공인 IP 반납]**
- 옛 서버 `yau-web-svr-01` : 정지 상태 확인 → **[서버 반납]**

✅ **결과 확인**: 새 IP로 SSH · 웹 접속 성공 + 목록에 옛 서버 · 옛 공인 IP 없음

📋 Day 4 실습 FAQ 총정리

단계	가장 흔한 문제	가장 빠른 확인 방법
Lab 00	접속이 안 됨	서버 **운영중**? 공인 IP가 기록 카드와 같은지?
Lab 01	Remote SSH 연결 실패	ACG **22** · 비밀번호 다시 복사 · 핫스팟
Lab 02	`claude: command not found`	`source ~/.bashrc` · 터미널 다시 열기
Lab 03	팀 조직이 안 보임	초대 수락 → `/logout` → `/login`
Lab 04	규칙 파일이 없음	`cat /root/day4-lab/CLAUDE.md` · Claude를 `/root/day4-lab` 에서 실행
Lab 05	웹 변경이 안 보임	강력 새로고침 · `cat` 으로 파일 확인
Lab 05	메모 저장 실패 · 502	`curl 127.0.0.1:5000/api/memos?date=2026-11-08` · API 실행 상태 확인
Lab 07	이미지가 안 보임	`Server > Server Image` 에서 상태 확인 (수 분 소요)
Lab 08	복구 후 요금이 두 배	옛 공인 IP · 옛 서버 **반납** 확인

📋 Day 4 실습 FAQ 총정리 — Lab 00~03 (확대 보기)

단계	가장 흔한 문제	가장 빠른 확인 방법
Lab 00	접속이 안 됨	서버 **운영중**? 공인 IP가 기록 카드와 같은지?
Lab 01	Remote SSH 연결 실패	ACG **22** · 비밀번호 다시 복사 · 핫스팟
Lab 02	`claude: command not found`	`source ~/.bashrc` · 터미널 다시 열기
Lab 03	팀 조직이 안 보임	초대 수락 → `/logout` → `/login`

📋 Day 4 실습 FAQ 총정리 — Lab 04~08 (확대 보기)

단계	가장 흔한 문제	가장 빠른 확인 방법
Lab 04	규칙 파일이 없음	`cat /root/day4-lab/CLAUDE.md` · Claude를 `/root/day4-lab` 에서 실행
Lab 05	웹 변경이 안 보임	강력 새로고침 · `cat` 으로 파일 확인
Lab 05	메모 저장 실패 · 502	`curl 127.0.0.1:5000/api/memos?date=2026-11-08` · API 실행 상태 확인
Lab 07	이미지가 안 보임	`Server > Server Image` 에서 상태 확인 (수 분 소요)
Lab 08	복구 후 요금이 두 배	옛 공인 IP · 옛 서버 **반납** 확인

🏆 Day 4 실습 달성 성과

- **오늘 여러분이 완수한 결과**:
  - ✅ VS Code Remote SSH / MobaXterm / 터미널로 서버 접속 · 파일 주고받기
  - ✅ 서버에 Claude Code 설치 · 팀 계정 로그인
  - ✅ `CLAUDE.md` 안전 규칙 · 권한 모드 이해
  - ✅ Claude Code로 **하루 할 일 메모 앱** 바이브 코딩 — Day 3 달력 연동 · **서버(`memos.json`)에 데이터 저장**
  - ✅ PC · 스마트폰 어디서 열어도 **같은 메모** · 새로고침해도 유지 (v4)
  - ✅ 서버 이미지 `yau-web-img-day4` 생성 · 오래된 이미지 정리
  - ⭐ (과제) 메모 앱 v5~v6 · GitHub에 작업물 Push
- **다음 수업 준비물**:
  - 📝 실습 기록 카드 (공인 IP · root 비밀번호 · 이미지 이름)
  - 🖥️ **반납하지 않은** 서버 + Claude Code 로그인 상태

🎉 수고하셨습니다! Day 4 실습 완료

AI 에이전트가 일하는 서버 환경을 직접 구축했습니다.

질문 및 트러블슈팅 Q&A 시간